

KristalBASIC v1.0 El Kitabı

KristalBASIC, C dilinin muazzam performansını (zero-cost abstraction) Basic dillerinin kolay ve okunaklı sözdizimiyle birleştiren modern, derlenen bir programlama dilidir. Bu el kitabı, KristalBASIC v1.0 sürümündeki tüm özellikleri, veri tiplerini, gömülü fonksiyonları ve oyun/grafik motoru entegrasyonlarını kapsar.

1. Temel Sözdizimi (Syntax) ve Yorum Satırları

KristalBASIC, satır sonlarında noktalı virgül (;) gerektirmez. Süslü parantezler ({ }) ile bloklar oluşturulur. Yorum satırları için # işareti kullanılır:

```
# Bu bir yorum satırıdır  
INT x = 10
```

2. Veri Tipleri ve Değişkenler

Değişken tanımlarken tipini açıkça belirtmeniz gerekir (Statik Tip Sistemi).

- INT:** Tamsayılar (INT yas = 25)
- DECIMAL:** Ondalıkli sayılar (DECIMAL pi = 3.14)
- STRING:** Metinler (STRING isim = "Kristal")
- BOOL:** Mantıksal ifadeler (BOOL aktif = TRUE, FALSE)

String Interpolation (Metin Birleştirme)

Değişkenleri metin içine \${degisken} formatında gömebilirsiniz:

```
STRING isim = "Ali"  
INT yas = 25  
print("Merhaba, benim adim ${isim} ve ${yas} yasındayım.\n")
```

3. Operatörler

- Matematiksel:** +, -, *, /, %
- Karşılaştırma:** ==, !=, <, >, <=, >=
- Mantıksal:** AND, OR, NOT

4. Kontrol Yapıları

IF / ELSE

```
IF (puan > 50) {  
    print("Gecti\n")  
} ELSE {  
    print("Kaldi\n")  
}
```

SWITCH / CASE

```
SWITCH (secim) {  
    CASE 1:  
        print("Birinci\n")  
    CASE 2:  
        print("Ikinci\n")  
    DEFAULT:  
        print("Diger\n")  
}
```

WHILE Döngüsü

```
INT i = 0  
WHILE (i < 10) {  
    print("Sayi: " + i + "\n")  
    i = i + 1  
}
```

FOR Döngüsü

```
FOR (INT i = 0 TO 10) {  
    print("Deger: " + i + "\n")  
}
```

5. Dinamik Veri Yapıları

LIST (Dinamik Diziler)

Listeler, aynı tipten birden fazla veriyi esnek bir şekilde tutar.

```
LIST INT sayilar = NEW LIST  
sayilar.append(10)  
sayilar.append(20)  
  
print("Eleman sayisi: " + sayilar.length() + "\n")  
print("Ilk eleman: " + sayilar.get(0) + "\n")  
  
# FOR IN ile Liste Gezinme  
FOR (INT s IN sayilar) {  
    print("Sayi: " + s + "\n")  
}
```

DICT (Sözlükler / Hash Map)

Anahtar-Değer (Key-Value) çiftlerini tutar.

```
DICT STRING INT yaslar = NEW DICT
yaslar.put("Ali", 25)
yaslar.put("Ayse", 30)

IF (yaslar.has("Ali")) {
    print("Ali'nin yasi: " + yaslar.get("Ali") + "\n")
}

yaslar.remove("Ali")
print("Sozluk boyutu: " + yaslar.length() + "\n")
```

6. Gelişmiş Tipler (OOP ve Yapılar)

ENUM (Numaralandırma)

Performans kaybı yaşatmayan (zero-cost) numaralandırmalar:

```
ENUM Durum {
    BEKLEMEDE,
    BASLADI = 10,
    BITTI
}

INT mevcut = Durum.BASLADI
```

STRUCT (Veri Yapıları)

C dilindeki gibi bellek dostu veri paketleri:

```
STRUCT Vektor2 {
    DECIMAL x
    DECIMAL y
}

Vektor2 v = NEW Vektor2
v.x = 10.5
v.y = 20.0
```

CLASS (Sınıflar)

Nesne Yönelimli Programlama (OOP) desteği:

```
CLASS Oyuncu {
    STRING isim
    INT can

    FUNC VOID HasarAl(INT miktar) {
        THIS.can = THIS.can - miktar
    }
}

Oyuncu p1 = NEW Oyuncu
p1.isim = "Kahraman"
p1.can = 100
p1.HasarAl(20)

# Obje ile işimiz bittiğinde belleği manuel olarak temizleyebiliriz
DELETE p1
```

7. Fonksiyonlar ve Modüller

Kendi Fonksiyonlarınızı Yazmak

```
FUNC INT Topla(INT a, INT b) {  
    RETURN a + b  
}
```

IMPORT (Modül Yükleme)

Farklı dosyalardaki kodları içeri aktarmak:

```
# utils.kbas dosyasını 'u' alias'ı ile yükler  
IMPORT "utils.kbas" AS u  
  
INT sonuc = u.Topla(5, 10)
```

8. Hata Yakalama (Try-Catch)

```
TRY {  
    # Riskli kod  
    IF (x == 0) {  
        THROW "Sifira bolme hatasi!"  
    }  
} CATCH (hata_mesaji) {  
    print("HATA: " + hata_mesaji + "\n")  
}
```

9. Metin (String) Metodları

- `isim.length()`: Karakter sayısını verir (INT).
- `isim.toUpperCase()`: Tüm harfleri büyütür.
- `isim.toLowerCase()`: Tüm harfleri küçültür.
- `isim.substring(baslangic, uzunluk)`: Metnin bir kısmını alır.
- `isim.replace(eski, yeni)`: Metin değiştirme.
- `isim.trim()`: Baş ve sondaki boşlukları siler.
- `isim.indexOf(aranan)`: Metin içinde arama yapar (Bulamazsa -1).
- `isim.split(ayrac)`: Metni parçalayıp LIST_STRING döndürür.

10. Dosya (I/O) İşlemleri

- `WRITE_FILE("log.txt", "Merhaba\n")`: Dosya oluşturur ve içine yazar.

- `APPEND_FILE("log.txt", "Ek satır\n")`: Dosyanın sonuna ekleme yapar.
 - `READ_FILE("log.txt")`: Dosya içeriğini `STRING` olarak okur.
 - `FILE_EXISTS("log.txt")`: Dosya var mı? (`BOOL`).
 - `FILE_SIZE("log.txt")`: Dosya boyutu (`INT` - byte cinsinden).
 - `DELETE_FILE("log.txt")`: Dosyayı siler (`BOOL` başarılı/başarısız).
-

11. Konsol İşlemleri

- `print("Mesaj")`: Ekrana çıktı verir. (Alt satıra geçmek için `\n` kullanın).
 - `INPUT()`: Kullanıcıdan `STRING` olarak girdi bekler.
-

12. Matematik Fonksiyonları

- `MIN(a, b)`, `MAX(a, b)`: En küçük ve en büyük değer.
 - `ABS(x)`: Mutlak değer.
 - `SQRT(x)`: Karekök.
 - `POW(taban, us)`: Üs alma.
 - `FLOOR(x)`, `CEIL(x)`: Aşağı/Yukarı yuvarlama.
 - `SIN(x)`, `COS(x)`, `TAN(x)`: Trigonometrik fonksiyonlar (Radyan).
 - `ASIN(x)`, `ACOS(x)`, `ATAN2(y, x)`: Ters trigonometrik fonksiyonlar.
 - `RAD(derece)`, `DEG(radyan)`: Radyan/Derece dönüşümü.
 - `CLAMP(deger, min, max)`: Değeri aralığa sıkıştırır.
 - `LERP(baslangic, bitis, miktar)`: İki değer arasında doğrusal geçiş yapar.
 - `RAND(min, max)`: Belirtilen aralıkta rastgele tamsayı üretir.
-

13. Grafik ve Oyun Motoru (Raylib Entegrasyonu)

Pencere Yönetimi

- `INIT_WINDOW(genislik, yukseklik, "Baslik")`: Pencere oluşturur.
- `SET_FPS(60)`: Kare hızını kilitler.
- `WINDOW_SHOULD_CLOSE()`: Kapanma tuşuna basıldı mı? (`BOOL`).
- `TOGGLE_FULLSCREEN()`: Oyunu tam ekran (fullscreen) moduna geçirir / çıkarır.

- `CLOSE_WINDOW( )`: Pencereyi kapatır.
- `CLEAR_BACKGROUND(HexRenk)`: Arkaplanı temizler (Örn: `0x181818`).
- `BEGIN_FPS( ) / DRAW_FPS(x, y)`: FPS gösterimi.
- `GET_TIME( )`: Geçen süreyi saniye cinsinden verir (DECIMAL).
- `FRAME_TIME( )`: Bir önceki kareden beri geçen süre (DeltaTime) (DECIMAL).

Girdi (Klavye & Fare)

- `KEY_DOWN(tus)`, `KEY_PRESSED(tus)`, `KEY_RELEASED(tus)`: Klavye kontrolleri (Örn: 'W').
- `MOUSE_DOWN(tus)`, `MOUSE_PRESSED(tus)`: Fare butonları (0: Sol, 1: Sağ, 2: Orta).
- `MOUSE_X( )`, `MOUSE_Y( )`: Farenin ekrandaki koordinatları (INT).
- `MOUSE_DELTA_X( )`, `MOUSE_DELTA_Y( )`: Farenin önceki kareye göre hareketi (DECIMAL).
- `MOUSE_WHEEL( )`: Fare tekerleği (DECIMAL).
- `HIDE_CURSOR( )`, `SHOW_CURSOR( )`, `DISABLE_CURSOR( )`: İmleç ayarları.

2D Çizimler

- `DRAW_CIRCLE(x, y, yarıcap, HexRenk)`
- `DRAW_RECTANGLE(x, y, genişlik, yükseklik, HexRenk)`
- `DRAW_LINE(x1, y1, x2, y2, HexRenk)`
- `DRAW_TEXT("Metin", x, y, boyut, HexRenk)`

3D Çizimler ve Vektörler

Vektör 3D yapıları performansı artırmak için gömülü VEC3 objesini kullanır.

- `VEC3(x, y, z)`: Yeni bir vektör tanımlar.
- `VEC3_LEN(v)`: Vektörün uzunluğu.
- `VEC3_NORM(v)`: Normalize edilmiş vektör.
- `VEC3_DOT(v1, v2)`: Vektörel çarpım (Dot product).

3D Dünyaya Geçiş:

- `BEGIN_3D(kamera_x, kamera_y, kamera_z, hedef_x, hedef_y, hedef_z)`: 3 Boyutlu çizim modunu başlatır.
- `END_3D( )`: 3D modu kapatır.

3D Çizim Fonksiyonları:

- `DRAW_CUBE(v_pos, genişlik, yükseklik, derinlik, HexRenk)`

- `DRAW_CUBE_WIRES(v_pos, genislik, yukseklik, derinlik, HexRenk)`
- `DRAW_SPHERE(v_pos, yaricap, HexRenk)`
- `DRAW_SPHERE_WIRES(v_pos, yaricap, HexRenk)`
- `DRAW_PLANE(v_pos, genislik, derinlik, HexRenk)`
- `DRAW_GRID(dilim, bosluk)`: Yere bir ızgara (Grid) çizer.
- `DRAW_CYLINDER(v_pos, ust_yaricap, alt_yaricap, yukseklik, HexRenk)`
- `DRAW_CAPSULE(v_baslangic, v_bitis, yaricap, HexRenk)`
- `DRAW_LINE3D(v_baslangic, v_bitis, HexRenk)`

Otomatik Arazi (Terrain) Sistemi

- `DRAW_TERRAIN(x_merkez, z_merkez, yaricap, yukseklik_carpani, HexRenk)`: 3D Perlin Noise tabanlı tepelik arazi çizer. (Yarıçap çizim mesafesini belirler).
- `TERRAIN_HEIGHT(x, z, yukseklik_carpani)`: Oyun karakterinin zemine basabilmesi için verilen x,z koordinatındaki arazi yüksekliğini döndürür (DECIMAL).

KristalBASIC v1.0 El Kitabı Sonu. *Kristalsoft tarafından Türkiye'de sevgiyle geliştirildi.*